Sesión-004

1. INTEGRIDAD DE DATOS (RESTRICCIONES CHECK)

¿Para qué sirve? Para ponerle candados de seguridad a los datos antes de que entren a la base de datos.

Explicación: Imagina que es un filtro en la puerta de un club. No importa quién seas (el programador, un administrador o un hacker), si intentas guardar un dato imposible (como "monedas negativas", "edad de -5 años" o un "nivel 0"), la base de datos te rechaza la operación y te tira un error.

En resumen: Sirve para garantizar que la información siempre tenga sentido lógico, incluso si el programa que está enviando los datos tiene un error (bug) o está hackeado.

```
-- Sesión 4: Integridad estructural y semántica
(anti-trampas a nivel de datos)
```

Comentario descriptivo del archivo. Las líneas que empiezan con -- no se ejecutan.

```
-- Motor: MySQL 8.0.16+ / MariaDB 10.2.1+
(restricciones CHECK aplicadas de verdad)
```

Aviso técnico: en versiones anteriores, MySQL aceptaba la sintaxis CHECK pero la ignoraba silenciosamente — hace falta una versión moderna para que realmente bloquee datos inválidos.

```
USE juego_online;
```

Selecciona la base sobre la que se van a modificar las tablas.

INTEGRIDAD SEMÁNTICA: VALORES IMPOSIBLES PARA UN JUGADOR

```
-- Integridad semántica: un jugador nunca puede
tener monedas o experiencia negativas,
```

Explica el objetivo: proteger el "significado" de los datos, no solo su forma (tipo de dato).

```
-- ni un nivel menor a 1 (evita que un "hack"
cliente-servidor deje datos inválidos).
```

Aclara el escenario real que se quiere prevenir: un cliente modificado que le pida al servidor un valor imposible.

```
ALTER TABLE Jugadores
```

ALTER TABLE significa "le voy a agregar algo a esta tabla que ya existe", en este caso a Jugadores.

```
ADD CONSTRAINT chk_monedas_no_negativas CHECK
(monedas >= 0),
```

Crea una restricción con nombre propio (chk_monedas_no_negativas): cualquier INSERT o UPDATE que deje monedas por debajo de 0 será rechazado por el motor, sin excepción.

```
ADD CONSTRAINT chk_experiencia_no_negativa CHECK (experiencia >= 0),
```

Misma idea para experiencia: nunca puede quedar en negativo.

```
ADD CONSTRAINT chk_nivel_minimo CHECK (nivel >= 1);
```

Misma idea para nivel: el mínimo posible es 1, nunca 0 ni negativo.

INTEGRIDAD ESTRUCTURAL: VALORES VÁLIDOS PARA UNA PARTIDA

```
-- Integridad estructural: un puntaje de partida no puede ser negativo,
```

Explica el objetivo de esta segunda tabla de reglas.

```
-- y el resultado debe ser uno de los valores válidos del dominio del juego.
```

Aclara que "resultado" no puede ser cualquier texto: solo un valor de una lista cerrada.

```
ALTER TABLE Partidas
```

Ahora se modifica la tabla Partidas.

```
ADD CONSTRAINT chk_puntaje_no_negativo CHECK (puntaje >= 0),
```

El puntaje de una partida nunca puede quedar en negativo.

```
ADD CONSTRAINT chk_resultado_valido CHECK (resultado IN ('victoria', 'derrota', 'jugada'));
```

El campo resultado solo acepta uno de estos tres textos exactos — cualquier otro valor (como 'empate' mal escrito o texto basura) es rechazado.

PRUEBA COMENTADA

```
-- Prueba: esto debe fallar (viola integridad semántica) — intento de "hackear" monedas negativas
```

Explica qué demuestra la siguiente línea comentada.

```
-- UPDATE Jugadores SET monedas = -100 WHERE jugador_id = 1;
```

Ejemplo (apagado, no se ejecuta) de un comando que el motor rechazaría con un error, gracias a chk_monedas_no_negativas. Puedes probarlo tú mismo quitando los -- del inicio.

2. ROLES Y PERMISOS (CONTROL DE ACCESOS)

¿Para qué sirve? Para que cada usuario solo pueda hacer exactamente lo que le corresponde hacer, ni más ni menos (Principio de Privilegio Mínimo).

Explicación: Es como un edificio de oficinas con tarjetas de acceso.

El Jugador tiene una tarjeta que solo le abre la puerta de "Jugar" y "Leer". No puede entrar a la sala de servidores (no puede modificar sus propias monedas).

El Moderador tiene una tarjeta que abre la puerta de "Banear", pero no la de "Asignar dinero".

El Administrador tiene la tarjeta maestra que abre todo.

En resumen: Sirve para proteger los datos confidenciales. Si la cuenta de un jugador es hackeada, el hacker no podrá destruir la base de datos ni darse monedas infinitas, porque ese usuario simplemente no tiene los permisos para hacerlo.

```
-- Sesión 4: Roles de aplicación — jugador, moderador, administrador
```

Comentario descriptivo del archivo.

```
-- Motor: MySQL 8.0+ / MariaDB 10.5+ (soporte de CREATE ROLE)
```

Aviso técnico: CREATE ROLE es relativamente reciente; versiones viejas de MySQL/MariaDB no lo soportan.

```
-- Requiere que la Sesión 1 (tablas y procedimiento subir_nivel_jugador) y la
```

Recuerda una dependencia entre sesiones.

```
-- Sesión 3 (procedimiento comprar_item_unico) ya se hayan ejecutado.
```

Continúa la aclaración: este script otorga EXECUTE sobre procedimientos que deben existir de antemano.

```
USE juego_online;
```

Selecciona la base sobre la que se crearán los roles y privilegios.

COLUMNA NUEVA PARA MODERACIÓN

```
-- Columna para banear jugadores (usada por moderadores)
```

Explica para qué sirve la columna que se va a agregar.

```
ALTER TABLE Jugadores ADD COLUMN IF NOT EXISTS baneado BOOLEAN NOT NULL DEFAULT FALSE;
```

Agrega el interruptor baneado a cada jugador, empezando en FALSE (nadie está baneado por defecto). Es la única columna que rol_moderador podrá modificar.

ROL JUGADOR — EL MÍNIMO NECESARIO PARA JUGAR

```
DROP ROLE IF EXISTS rol_jugador;
```

Borra el rol si ya existía, para poder recrearlo sin error.

```
CREATE ROLE rol_jugador;
```

Crea un rol — un "paquete" de privilegios con nombre, que después se puede asignar a cuentas reales.

```
GRANT SELECT ON juego_online.Jugadores TO
rol_jugador;
```
Puede leer la tabla de jugadores (su propio perfil y el de otros, para rankings), pero no modificarla directamente.
```
GRANT SELECT ON juego_online.Items TO rol_jugador;
```
Puede ver el catálogo de items disponibles.
```
GRANT SELECT ON juego_online.Juegos TO rol_jugador;
```
Puede ver el catálogo de juegos.
```
GRANT SELECT, INSERT ON juego_online.Partidas TO
rol_jugador;
```
Puede ver partidas y crear registros nuevos — así se anota cuando juega una partida.
```
GRANT EXECUTE ON PROCEDURE
juego_online.subir_nivel_jugador TO rol_jugador;
```
Puede ejecutar el procedimiento de la Sesión 1, que es la ÚNICA forma permitida de subir de nivel.
```
GRANT EXECUTE ON PROCEDURE
juego_online.comprar_item_unico TO rol_jugador;
```
Puede ejecutar el procedimiento de la Sesión 3, la ÚNICA forma permitida de comprar el item único.
```
-- Explícitamente NO se otorga UPDATE directo sobre
Jugadores.monedas/nivel
```
El punto clave de todo el diseño: como nunca se le da UPDATE sobre esas columnas, un jugador no puede asignarse monedas o nivel él mismo, ni siquiera con un cliente modificado — solo puede pasar por los procedimientos, que sí validan las reglas del juego.

ROL MODERADOR — HEREDA JUGADOR, PUEDE BANEAR

```
DROP ROLE IF EXISTS rol_moderador;
```
Borra el rol si ya existía.
```
CREATE ROLE rol_moderador;
```
Crea el rol moderador.
```
GRANT rol_jugador TO rol_moderador;
```
Esta línea es la clave de la jerarquía: le da a rol_moderador TODOS los privilegios de rol_jugador automáticamente, sin tener que repetirlos uno por uno.
```
GRANT UPDATE (baneado) ON juego_online.Jugadores TO
rol_moderador;
```
Además de lo heredado, le da permiso de UPDATE pero solo sobre la columna baneado — no puede tocar monedas, nivel ni ninguna otra columna con este privilegio.

ROL ADMINISTRADOR — HEREDA MODERADOR, CONTROLA MONEDAS

```
DROP ROLE IF EXISTS rol_administrador;
```

Borra el rol si ya existía.

```
CREATE ROLE rol_administrador;
```

Crea el rol administrador.

```
GRANT rol_moderador TO rol_administrador;
```

Hereda todo lo de rol_moderador (que a su vez ya incluye todo lo de rol_jugador) — una cadena de herencia de tres niveles.

```
GRANT UPDATE (monedas, nivel, experiencia) ON
juego_online.Jugadores TO rol_administrador;
```

Es el ÚNICO rol con permiso para modificar monedas, nivel y experiencia directamente con UPDATE, sin pasar por un procedimiento.

```
GRANT DELETE ON juego_online.Jugadores TO
rol_administrador;
```

Puede eliminar cuentas de jugadores.

```
GRANT DELETE ON juego_online.Partidas TO
rol_administrador;
```

Puede eliminar registros de partidas (por ejemplo, para anular resultados de un torneo con evidencia de trampa).

APLICAR, EJEMPLO DE USO Y VERIFICACIÓN

```
FLUSH PRIVILEGES;
```

Recarga la tabla interna de privilegios para que todos los cambios queden activos de inmediato.

```
-- Ejemplo: asignar el rol a una cuenta real y
activarlo por defecto
```

Introduce un ejemplo (comentado) de cómo se usaría esto con una cuenta de verdad.

```
-- CREATE USER 'moderador1'@'%' IDENTIFIED BY
'ClaveSegura123';
```

Crearía una cuenta real de MySQL para un moderador.

```
-- GRANT rol_moderador TO 'moderador1'@'%';
```

Le asignaría el rol (el paquete de privilegios) a esa cuenta.

```
-- SET DEFAULT ROLE rol_moderador TO
'moderador1'@'%';
```

Haría que el rol se active automáticamente cada vez que esa cuenta inicia sesión, sin tener que activarlo a mano con SET ROLE.

```
-- SHOW GRANTS FOR rol_administrador;
```

Comando (comentado) para listar todos los privilegios reales de este rol, incluidos los heredados.

3. ANTI-INYECCIÓN SQL (SEGURIDAD EN EL CÓDIGO)

¿Para qué sirve? Para evitar que un hacker "secuestre" tu base de datos metiendo código malicioso a través de los cuadros de texto (como un buscador o un login).

Explicación: Si tú programas un buscador y "pegas" el texto que escribe el usuario directamente en tu comando, un hacker puede escribir cosas como ' OR '1'='1 para engañar al sistema y robar TODA la información. La anti-inyección sirve para decirle a la base de datos: "Oye, esto que escribió el usuario es solo un texto normal, no lo ejecutes como si fuera una orden".

En resumen: Sirve para blindar tu aplicación y evitar que te roben, borren o modifiquen la información a través de los formularios de tu página web.

```
-- Sesión 4: Simulador — identificar y frenar inyecciones SQL (MySQL / MariaDB)
```

Comentario descriptivo del archivo.

```
-- Reto: un "hacker" intenta usar el buscador de jugadores por nombre para robar items
```

Plantea el escenario educativo de esta sesión.

```
-- o alterar las notas de un torneo, inyectando SQL dentro del campo de búsqueda.
```

Continúa el planteamiento del reto.

```
USE juego_online;
```

Selecciona la base sobre la que se creará el procedimiento seguro.

VERSIÓN VULNERABLE (DOCUMENTADA, NUNCA SE EJECUTA)

```
-- Si el backend arma la consulta concatenando texto del usuario, así:
```

Introduce el error de diseño que se quiere evitar: pegar texto del usuario directamente dentro de una cadena SQL.

```
-- texto_busqueda = "' OR '1'='1"
```

Ejemplo de lo que un atacante escribiría en el campo de búsqueda.

```
-- sql = "SELECT * FROM Jugadores WHERE nombre_usuario = '" + texto_busqueda + "'"
```

Así es como un backend mal construido armaría la consulta: pegando texto sin validar.

```
-- La consulta final ejecutada sería:
```

Muestra el resultado de esa concatenación.

```
-- SELECT * FROM Jugadores WHERE nombre_usuario = '' OR '1'='1'
```

La comilla del atacante cerró el texto vacío antes de tiempo, y OR '1'='1' agregó una condición que siempre es verdadera.

```
-- Esto devuelve TODOS los jugadores, filtrando la condición de búsqueda.
```
Explica el impacto: la búsqueda deja de filtrar por nombre y expone toda la tabla.
```
-- Un payload más dañino podría ser:
```
Introduce un segundo ejemplo, más peligroso.
```
-- texto_busqueda = "'; UPDATE Jugadores SET monedas = 999999 WHERE nombre_usuario = 'NoobMaster'; --"
```
Si además el backend permitiera varias sentencias por llamada, este texto no solo buscaría: también modificaría datos.
```
-- que otorgaría monedas infinitas al atacante si el backend permite múltiples sentencias.
```
Aclara la condición para que ese segundo ejemplo funcione: depende de que el conector permita varias sentencias separadas por punto y coma.

VERSIÓN SEGURA: CREAR EL PROCEDIMIENTO

```
DROP PROCEDURE IF EXISTS buscar_jugador_seguro;
```
Borra una versión anterior si existía, para poder recrearlo sin error.
```
DELIMITER //
```
Cambia temporalmente el separador de sentencias, porque el cuerpo del procedimiento tiene su propio ; interno.
```
CREATE PROCEDURE buscar_jugador_seguro(IN p_nombre_usuario VARCHAR(50))
```
Crea el procedimiento con un único parámetro de entrada: el nombre a buscar.
```
BEGIN
```
Empieza el cuerpo del procedimiento.
```
-- p_nombre_usuario siempre se trata como VALOR, nunca como código SQL,
```
La idea central de toda la defensa: el parámetro nunca se "pega" como texto dentro del SQL.
```
-- sin importar qué texto contenga (comillas, ; --, OR '1'='1', etc.)
```
Refuerza que esto es válido sin importar qué caracteres especiales traiga el texto.
```
SELECT jugador_id, nombre_usuario, nivel, monedas
```
Columnas que devuelve la búsqueda.
```
FROM Jugadores
```
Tabla consultada.
```
WHERE nombre_usuario = p_nombre_usuario;
```

Compara el nombre guardado contra el parámetro recibido como un simple valor — el motor nunca lo reinterpreta como sintaxis SQL.

```
END //
```

Cierra el cuerpo del procedimiento.

```
DELIMITER ;
```

Devuelve el delimitador normal (;) para las siguientes instrucciones.

PROBAR EL RETO

```
CALL buscar_jugador_seguro('\' OR \'1\'=\'1');
```

Llama al procedimiento pasándole literalmente el texto ' OR '1'='1 como nombre de usuario a buscar (las barras invertidas solo escapan las comillas dentro de este script de ejemplo).

```
-- Resultado esperado: 0 filas (porque ningún
jugador se llama literalmente ' OR '1'='1 )
```

Explica por qué no hay fuga de datos: el motor busca un jugador con ESE nombre exacto, que no existe.

```
CALL buscar_jugador_seguro('DragonSlayer99');
```

Prueba de control: una búsqueda normal, con un nombre real.

```
-- Resultado esperado: 1 fila, la del jugador real.
```

Confirma que el procedimiento sigue funcionando normalmente para búsquedas legítimas.

LECCIÓN FINAL

```
-- Lección: en el backend de la aplicación, usar
SIEMPRE consultas preparadas
```

Cierra el script con la recomendación general, más allá de este ejercicio puntual.

```
-- (prepared statements / placeholders ?, :nombre) y
nunca concatenar texto del
```

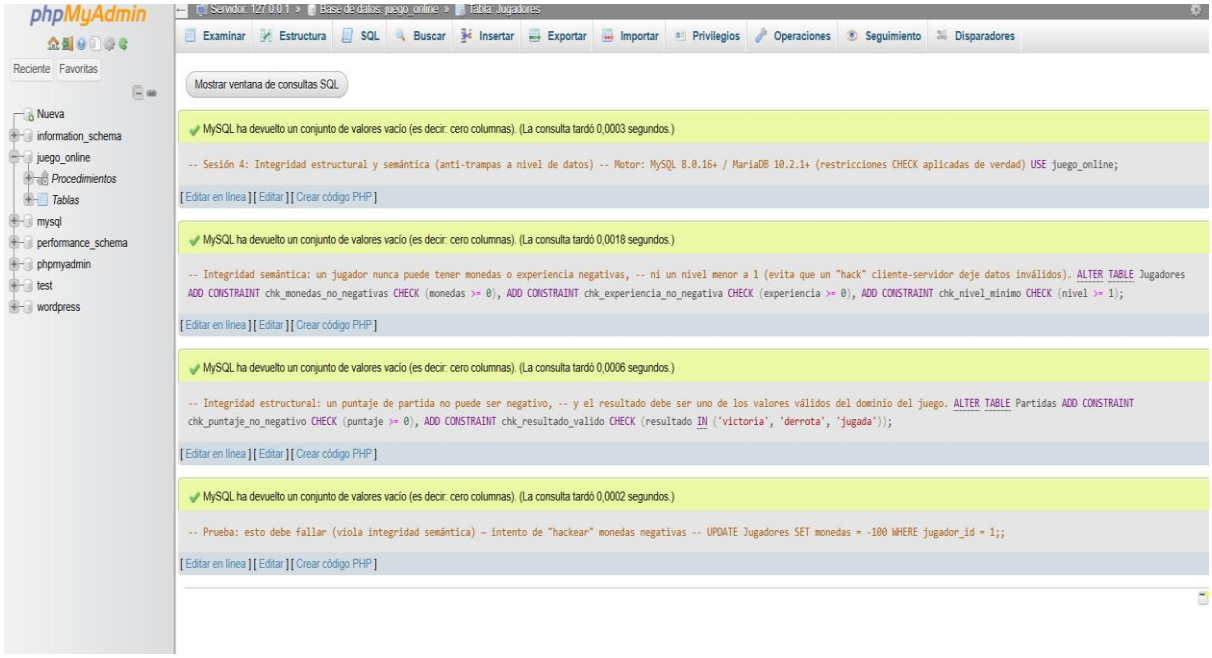
Nombra las herramientas concretas que existen en la mayoría de lenguajes para lograr esto.

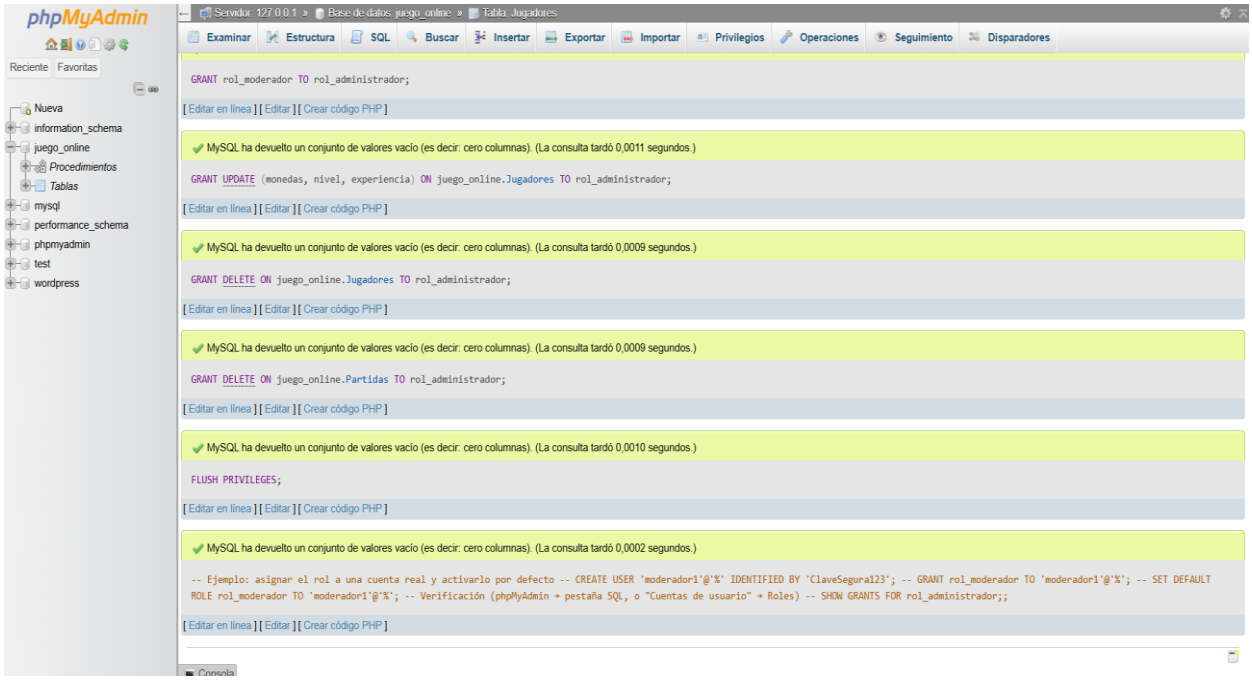
```
-- usuario directamente dentro de una sentencia SQL.
Este mismo principio es el
```

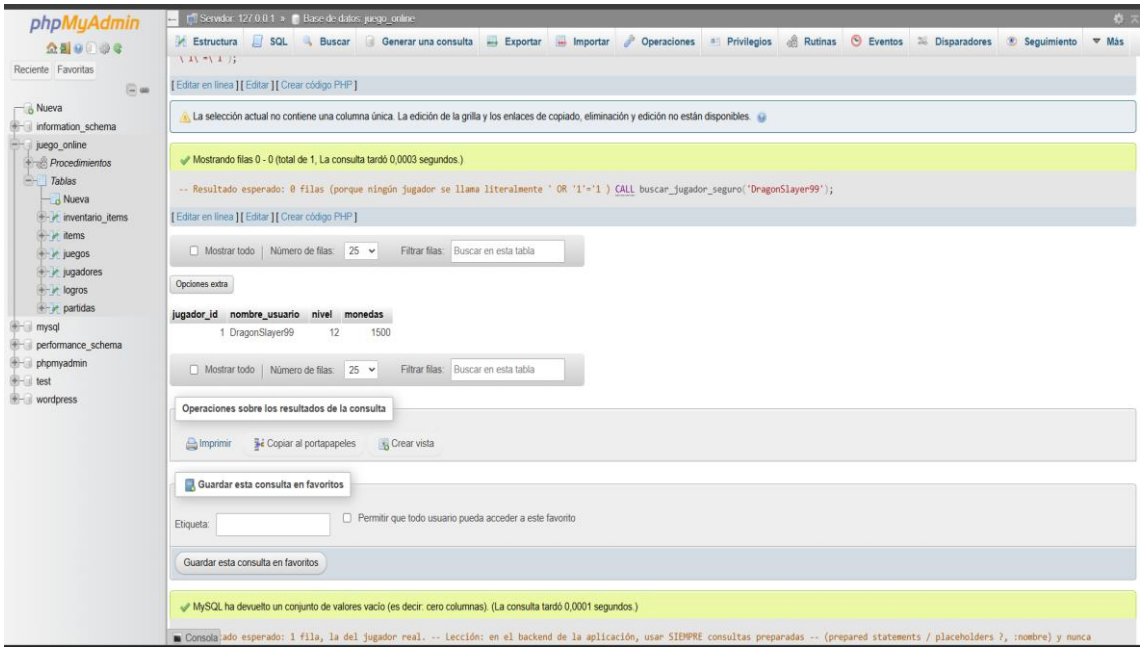
Refuerza que el mecanismo (parámetro separado del texto SQL) es universal.

```
-- que aplica un ORM o un driver con parámetros
vinculados (bound parameters).
```

Aclara que usar un ORM no es magia: por debajo, sigue dependiendo de este mismo principio de parámetros vinculados.


phpMyAdmin
Reciente Favoritas
Nueva
information_schema
juego_online
Procedimientos
Tablas
mysql
performance_schema
phpmyadmin
test
wordpress
Servidor: 127.0.0.1 » Base de datos: juego_online » Tabla: Jugadores
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores
Mostrar ventana de consultas SQL
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0003 segundos.)
-- Sesión 4: Integridad estructural y semántica (anti-trampas a nivel de datos) -- Motor: MySQL 8.0.16+ / MariaDB 10.2.1+ (restricciones CHECK aplicadas de verdad) USE juego_online;
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0018 segundos.)
-- Integridad semántica: un jugador nunca puede tener monedas o experiencia negativas, -- ni un nivel menor a 1 (evita que un "hack" cliente-servidor deje datos inválidos). ALTER TABLE Jugadores ADD CONSTRAINT chk_monedas_no_negativas CHECK (monedas >= 0), ADD CONSTRAINT chk_experiencia_no_negativa CHECK (experiencia >= 0), ADD CONSTRAINT chk_nivel_minimo CHECK (nivel >= 1);
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0006 segundos.)
-- Integridad estructural: un puntaje de partida no puede ser negativo, -- y el resultado debe ser uno de los valores válidos del dominio del juego. ALTER TABLE Partidas ADD CONSTRAINT chk_puntaje_no_negativo CHECK (puntaje >= 0), ADD CONSTRAINT chk_resultado_valido CHECK (resultado IN ('victoria', 'derrota', 'jugada'));
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0002 segundos.)
-- Prueba: esto debe fallar (viola integridad semántica) — intento de "hackear" monedas negativas -- UPDATE Jugadores SET monedas = -100 WHERE jugador_id = 1;;
[Editar en línea] [Editar] [Crear código PHP]


phpMyAdmin
Reciente Favoritas
Nueva
information_schema
juego_online
Procedimientos
Tablas
mysql
performance_schema
phpmyadmin
test
wordpress
Servidor: 127.0.0.1 » Base de datos: juego_online » Tabla: Jugadores
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores
GRANT rol_moderador TO rol_administrador;
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0011 segundos.)
GRANT UPDATE (monedas, nivel, experiencia) ON juego_online.Jugadores TO rol_administrador;
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0009 segundos.)
GRANT DELETE ON juego_online.Jugadores TO rol_administrador;
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0009 segundos.)
GRANT DELETE ON juego_online.Partidas TO rol_administrador;
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0010 segundos.)
FLUSH PRIVILEGES;
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0002 segundos.)
-- Ejemplo: asignar el rol a una cuenta real y activarlo por defecto -- CREATE USER 'moderador1'@'%' IDENTIFIED BY 'ClaveSegura123'; -- GRANT rol_moderador TO 'moderador1'@'%'; -- SET DEFAULT ROLE rol_moderador TO 'moderador1'@'%'; -- Verificación (phpMyAdmin → pestaña SQL, o "Cuentas de usuario" → Roles) -- SHOW GRANTS FOR rol_administrador;;
[Editar en línea] [Editar] [Crear código PHP]
Consola


phpMyAdmin
Reciente Favoritas
Nueva
information_schema
juego_online
Procedimientos
Tablas
Nueva
inventario_items
items
juegos
jugadores
logros
partidas
mysql
performance_schema
phpmyadmin
test
wordpress
Servidor: 127.0.0.1 » Base de datos: juego_online
Estructura SQL Buscar Generar una consulta Exportar Importar Operaciones Privilegios Rutinas Eventos Disparadores Seguimiento Más
[Editar en línea] [Editar] [Crear código PHP]
La selección actual no contiene una columna única. La edición de la grilla y los enlaces de copiado, eliminación y edición no están disponibles.
Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0003 segundos.)
-- Resultado esperado: 0 filas (porque ningún jugador se llama literalmente ' OR '1'='1) CALL buscar_jugador_seguro('DragonSlayer99');
[Editar en línea] [Editar] [Crear código PHP]
Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla
Opciones extra
jugador_id nombre_usuario nivel monedas
1 DragonSlayer99 12 1500
Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla
Operaciones sobre los resultados de la consulta
Imprimir Copiar al portapapeles Crear vista
Guardar esta consulta en favoritos
Etiqueta:
Permitir que todo usuario pueda acceder a este favorito
Guardar esta consulta en favoritos
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0001 segundos.)
Consola
ado esperado: 1 fila, la del jugador real. -- Lección: en el backend de la aplicación, usar SIEMPRE consultas preparadas -- (prepared statements / placeholders ?, :nombre) y nunca